



FACULTAD DE INGENIERÍA

Machine Learning en Producción

Obligatorio: Predicción de ratings de películas IMDb

2026

Alumnos:

Pablo Durán - 270956

Marcos Mussio - 362839

Natalia Campiglia - 349251



[mlp-obligatorio-imdb](#)

Última actualización: 9/07/2026

Índice

1. Resumen	4
2. Objetivo del proyecto	4
3. Arquitectura general	5
4. Recolección de datos	7
4.1. Ingesta de datos	7
4.2. Almacenamiento y versionado de datos	9
4.3. ETL	9
4.4. Logs y diagnóstico del proceso	11
4.5. Limitantes	12
5. Entrenamiento	14
5.1. Feature engineering	14
5.1.1. Feature engineering del entrenamiento actual	15
5.2. Entrenamiento del modelo	16
5.3. Publicación en W&B	17
5.4. Prevención de data leakage y training-serving skew	18
5.4.1. Data leakage	18
5.4.2. Training-serving skew	19
5.5. Trazabilidad de ML	20
6. Servicio/API	21
6.1. API de inferencia	21
6.2. Predicciones en landing page	21
6.3. Predicciones batch	24
6.4. Containerización	25
7. Infraestructura y monitoreo	26
7.1. Infraestructura como código	26

7.2. Gestión de secretos	28
7.3. Observabilidad y monitoreo	29
8. Limitaciones del proyecto y arquitectura ideal	32
8.1. Limitantes	32
8.2. Arquitectura ideal	33
9. Conclusiones	35

1. Resumen

El proyecto implementa una solución de Machine Learning en Producción para predecir el rating de películas de IMDb. La solución cubre el ciclo completo desde la recolección de datos hasta el despliegue de una aplicación web en AWS, donde es flujo principal se muestra en la siguiente imagen.

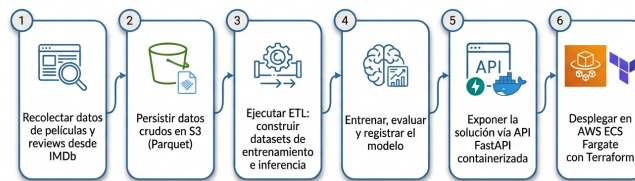


Figura 1: Flujo principal

La solución busca aplicar conceptos de MLOps vistos en clase, automatización de pipelines, versionado de datos, separación entre datos crudos y procesados, tracking de modelos, model registry, despliegue reproducible, gestión de secretos y monitoreo básico. Estas prácticas son relevantes porque los sistemas de Machine Learning suelen acumular deuda técnica no solo en el modelo, sino también en los datos, pipelines, configuración y serving [4, 3].

2. Objetivo del proyecto

El objetivo es construir una aplicación capaz de estimar el rating esperado de una película a partir de atributos disponibles en IMDb, como año, duración, metascoring, géneros, director, elenco principal y señales derivadas de reviews de usuarios.

Además del modelo predictivo, el foco del obligatorio está en construir una arquitectura productiva alrededor del modelo. Por eso el proyecto no se limita al entrenamiento, sino que incluye ingesta de datos, procesamiento, almacenamiento, despliegue cloud e infraestructura como código.

3. Arquitectura general

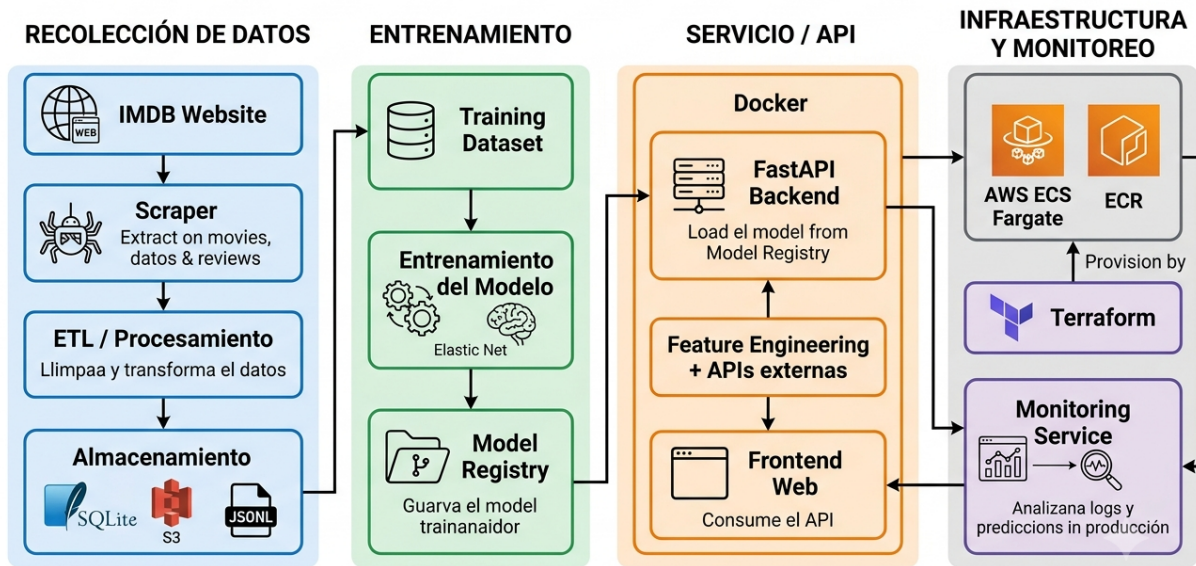


Figura 2: Arquitectura general de la solución: recolección de datos, entrenamiento, servicio/API e infraestructura cloud.

La Figura 2 resume la solución de punta a punta. El flujo comienza con la extracción de información desde IMDb mediante un scraper desarrollado en Python con Playwright. Ese scraper recolecta datos de películas y reviews, y los persiste en S3 como datos crudos, separados por tipo de entidad y por corrida. Esta primera capa funciona como data lake y permite conservar el historial de extracciones sin sobrescribir resultados anteriores.

Sobre esos datos crudos se ejecuta el proceso ETL, implementado con pandas y pyarrow. El ETL consolida las corridas del scraper, deduplica películas, agrega la señal de sentimiento de reviews y genera los datasets procesados que luego consumen el entrenamiento y la inferencia. De esta forma, la arquitectura separa explícitamente la capa raw de la capa procesada, lo que mejora la trazabilidad y facilita repetir el procesamiento si cambia la lógica de transformación.

Luego, el pipeline de entrenamiento toma el dataset procesado, aplica feature engineering y entrena el modelo predictivo. Los experimentos, métricas y artefactos del modelo se registran en Weights & Biases, que cumple el rol de tracking y model registry. Desde allí, la aplicación de serving puede recuperar el modelo productivo usando el alias correspondiente.

La parte de serving está implementada con FastAPI y se empaqueta en una imagen Docker. Esa imagen se publica en ECR y se despliega en AWS ECS Fargate detrás de un Application Load Balancer. La infraestructura se define con Terraform, lo que permite recrear el entorno de manera declarativa. Finalmente, los logs de ejecución quedan disponibles en CloudWatch, completando una primera capa de observabilidad para la API desplegada.

4. Recolección de datos

El proceso de recolección y preparación inicial de datos se ejecutó localmente. Para hacerlo repetible, se creó un `Makefile` que agrupa los comandos principales de setup, credenciales, login en IMDb, ejecución del scraper y corrida del ETL. Esto evita depender de comandos manuales largos y permite correr el flujo completo de forma más ordenada.

Comando	Uso
<code>make install</code>	Instala las dependencias del proyecto con Poetry y descarga Chromium para Playwright.
<code>make aws-credentials</code>	Copia las credenciales del Learner Lab desde el portapapeles y las guarda en <code>~/.aws/credentials</code> .
<code>make scraper-login</code>	Abre el navegador para realizar el login manual en IMDb y guardar la sesión local.
<code>make scraper-run</code>	Ejecuta el scraper y sube los datos crudos a S3 en formato Parquet.
<code>make etl-run</code>	Lee los datos crudos desde S3, los procesa y publica el dataset de entrenamiento.
<code>make scraper-etl</code>	Ejecuta en orden el scraper y luego el ETL.
<code>make pipeline</code>	Ejecuta el flujo completo: credenciales, login, scraper y ETL.

Esta automatización fue especialmente útil porque el scraping requiere pasos locales, como la autenticación manual en IMDb, y pasos cloud, como la escritura de datos en S3 con credenciales temporales de AWS Academy/Learner Lab que expiran y deben renovarse periódicamente. El `Makefile` deja explícito el orden esperado, reduce errores operativos al repetir corridas y hace que el proceso sea más simple.

4.1. Ingesta de datos

La ingesta se realiza con un scraper de IMDb implementado en Python. El scraper obtiene información de películas y reviews, y guarda los resultados en formato local y en S3.

El scraper no parte de una lista fija de películas. En cada corrida genera IDs aleatorios dentro

de un rango configurado, con el formato propio de IMDb, por ejemplo `tt00000001`. Con cada ID arma una URL de título:

```
https://www.imdb.com/title/{imdb_id}/
```

Luego visita esa página con Playwright, valida que el contenido corresponda a una película y extrae los atributos principales. Si el ID corresponde a otro tipo de contenido o no se encuentra título, se descarta y se continúa con otro ID aleatorio.

Las reviews se obtienen desde una URL aparte. Primero el scraper busca en la página de la película el enlace a la sección de reviews de usuarios; si no lo encuentra, usa por defecto:

```
https://www.imdb.com/title/{imdb_id}/reviews/
```

Esa página de reviews está paginada. Por eso el scraper lee las tarjetas visibles, expande spoilers cuando corresponde y luego intenta avanzar con el botón de “load more” hasta alcanzar el máximo configurado de reviews por película o hasta que ya no haya más páginas disponibles. Si no logra obtener reviews desde la página completa, usa como fallback las reviews destacadas que aparecen dentro de la página principal de la película.

Los datos recolectados se separan en dos entidades principales:

Entidad	Campos recolectados
Películas	Identificador IMDb, título, año, rating IMDb, cantidad de votos, géneros, director/es, reparto principal, duración, certificado de edad, metascore y sinopsis.
Reviews	Película asociada, usuario, rating de la review, texto de la review y votos de utilidad.

El scraper contempla que IMDb puede requerir login para acceder a reviews completas. Para eso existe un flujo manual que guarda una sesión local y permite reutilizarla en corridas posteriores.

4.2. Almacenamiento y versionado de datos

Los datos se almacenan en S3 en formato Parquet. Se eligió Parquet porque es un formato columnar, eficiente para lectura y escritura de datos tabulares, y se integra bien con pandas, pyarrow y servicios cloud como S3. Además, comprime mejor que formatos de texto como CSV o JSON y conserva tipos de datos de forma más robusta, algo útil para una capa raw que luego será procesada por el ETL.

Cada corrida del scraper genera archivos con timestamp, evitando sobrescribir datos anteriores.

```
s3://mlp-imdb-data/  
imdb/  
  movies/  
    scraped_date=YYYY-MM-DD/  
      run_YYYYMMDD_HHMMSS.parquet  
  reviews/  
    scraped_date=YYYY-MM-DD/  
      run_YYYYMMDD_HHMMSS.parquet  
  processed/  
    training_dataset.json  
  inference/  
    inference_dataset.json
```

Esta organización permite aplicar un concepto importante de MLOps: trazabilidad de datos. Si un modelo fue entrenado con determinada versión del dataset, se puede identificar de qué corridas del scraper provienen los datos.

4.3. ETL

El proceso ETL transforma los datos crudos del scraper en un dataset consolidado por película. En la implementación actual, el ETL lee datos desde S3, deduplica películas, agrega una señal de sentimiento a partir de reviews y genera un JSON procesado que luego consume el entrenamiento.

Los pasos principales son:

1. Validar que existan objetos en los prefijos `movies/` y `reviews/` de S3.
2. Leer todos los Parquet crudos de películas y reviews desde S3.
3. Deduplicar películas repetidas por `imdb_id`, conservando la versión con `scraped_date` más reciente.
4. Calcular `review_agg`, una tabla agregada de sentimiento promedio por película.
5. Unir películas con `review_agg` mediante `imdb_id`.
6. Eliminar filas sin `imdb_rating`, porque no pueden usarse para entrenamiento supervisado.
7. Generar registros limpios con tipos JSON válidos, rating normalizado a escala 0–1 y listas normalizadas.
8. Guardar `training_dataset.json` localmente y subirlo a S3:

`s3://mlp-imdb-data/imdb/processed/training_dataset.json`

Una de las transformaciones del ETL es generar `review_agg`. Como el scraper guarda varias reviews por película, el ETL resume esos textos en una única señal por `imdb_id`.

Para eso se aplica VADER, a través de la librería `vaderSentiment`. Se eligió porque es liviano, reproducible y no requiere entrenar un modelo adicional. VADER asigna un score de polaridad a cada review, y luego el ETL calcula el promedio por película.

Finalmente, esa señal agregada se une al dataset de películas. Si una película no tiene reviews válidas, simplemente queda sin esa feature y el pipeline de entrenamiento se encarga de imputarla.

El resultado del ETL es un JSON procesado con una fila por película y los campos necesarios para entrenamiento:

Campo del JSON procesado	Descripción
<code>imdb_id</code>	Identificador de IMDb; se conserva para trazabilidad, pero no entra al modelo.
<code>title</code>	Título de la película.
<code>year</code>	Año de estreno.
<code>votes</code>	Cantidad de votos.
<code>directors</code>	Lista normalizada de directores.
<code>main_cast</code>	Lista normalizada de actores principales.
<code>reviews</code>	Sentimiento promedio de reviews calculado con VADER y normalizado a escala 0–1.
<code>imdb_rating</code>	Variable objetivo, normalizada a escala 0–1.

4.4. Logs y diagnóstico del proceso

Tanto el scraper como el ETL incorporan logs para seguir la corrida completa y detectar problemas temprano. En el scraper se usa un logger propio que escribe tanto en consola como en archivos diarios dentro de `data/logs/`. Al comenzar, registra el rango de IDs aleatorios que va a intentar, la cantidad máxima de películas y la cantidad máxima de reviews por película. Esto permite reproducir el contexto de una corrida y entender si el volumen obtenido coincide con la configuración esperada.

Durante la autenticación, los logs indican si se reutilizó una sesión guardada, si fue necesario iniciar sesión nuevamente o si el login falló. Esto es importante porque, cuando IMDb bloquea el acceso a la página completa de reviews, el scraper puede quedar limitado a reviews destacadas. Registrar ese estado ayuda a explicar por qué una corrida puede traer menos reviews que otra.

Mientras recorre títulos, el scraper informa cuando descarta un ID porque no corresponde a una película, cuando no encuentra título, o cuando ocurre un error al extraer una película o sus reviews. También registra el progreso de guardado con el ID de IMDb, la cantidad de películas acumuladas y cuántas reviews se obtuvieron. Al final imprime una tabla con las películas guardadas

ordenadas por cantidad de reviews y cuenta cuántas películas tienen rating pero no reviews. Esta salida sirve para análisis rápido: permite detectar películas sin señal textual, problemas de acceso a reviews o una muestra demasiado pobre para el ETL.

Finalmente, si S3 está configurado, el scraper registra la subida de los Parquet crudos con la cantidad de películas, cantidad de reviews y `run_id`. Si falla el upload, el log lo deja explícito, lo que facilita distinguir entre un problema de scraping y un problema de credenciales o conectividad con AWS. La ventaja de subir datos scrapeados en formato Parquet es que si en el futuro se desea conectar el bucket a un Data Warehouse (ej Redshift, Snowflake, Databricks SQL) mapeando dicho bucket a una tabla externa, esto permite poder resolver queries en forma más performante ya que Parquet ayuda con la compresión de los datos.

El ETL incluye varios mensajes de diagnóstico para detectar problemas temprano y entender la calidad de los datos antes de entrenar. Primero valida que existan archivos en los prefijos de S3 para películas y reviews; si faltan datos o credenciales, muestra un mensaje específico para saber si el problema viene de AWS, del bucket o de una corrida pendiente del scraper.

Durante la lectura imprime la cantidad de filas crudas de películas y reviews. Después muestra ejemplos útiles para análisis exploratorio: las 10 películas con mayor rating, las 10 con menor rating y las primeras reviews crudas. Estos logs permiten revisar rápidamente si los datos parecen razonables, si hay ratings fuera de escala o si las reviews llegan con los campos esperados.

Luego de deduplicar, el ETL informa cuántas películas quedan y muestra un resumen de nulos por dataset. Esto ayuda a identificar columnas incompletas, problemas del scraper o señales que luego habrá que imputar. Al final, informa cuántas filas sin `imdb_rating` se eliminaron, cuántas películas quedan en el dataset final y si todavía existen columnas con nulos. Con esa información se pueden sacar conclusiones simples antes de entrenar, por ejemplo si el dataset tiene pocas reviews útiles, si hay muchos registros sin target o si la extracción necesita repetirse.

4.5. Limitantes

Durante la recolección aparecieron algunas limitantes operativas. La primera fue que el scraping de reviews requería iniciar sesión en IMDb para acceder a la página completa de comentarios.

Por eso se incorporó un paso de login manual y reutilización de sesión local; sin esa sesión, el scraper podía quedar limitado a las reviews destacadas disponibles en la página principal de la película.

Otra limitante fue la duración de las credenciales temporales de AWS Academy/Learner Lab. Como esas credenciales vencen en cuestión de algunas horas, las corridas se acotaron a volúmenes manejables, por ejemplo de a 500 reviews. Cuando se intentaba aumentar demasiado el volumen de scraping, podía pasar que al momento de guardar los datos en S3 las credenciales ya hubieran expirado, obligando a renovar credenciales y repetir parte del proceso.

Por último, al trabajar con una cuenta de Amazon Academy, no se logró dejar los datos de S3 accesibles para otros compañeros de forma simple. Esto limitó el uso compartido directo del bucket y reforzó la necesidad de documentar bien las rutas, corridas y archivos generados.

5. Entrenamiento

Esta sección describe el flujo completo de entrenamiento del modelo: cómo se transforman los datos procesados en features, cómo se entrena y evalúa el modelo, cómo se publica el artefacto en W&B y qué controles se consideran para evitar problemas habituales en sistemas de Machine Learning en Producción.

5.1. Feature engineering

El feature engineering del modelo ocurre principalmente en `training/train.py`, después de cargar `training_dataset.json`. El ETL deja los datos limpios y agregados; el entrenamiento convierte esos campos en representaciones numéricas aptas para scikit-learn.

Uso de columnas en el entrenamiento actual:

Columna	Uso actual o motivo
<code>imdb_id</code>	Se excluye del entrenamiento porque es un identificador, no una señal predictiva.
<code>imdb_rating</code>	Se excluye de las features porque es el target.
<code>votes</code>	No se excluye en el modelo actual: se transforma a <code>log_votes</code> . Puede ser una fuente de leakage si se quiere simular una predicción previa al estreno o muy temprana, pero es válida para el escenario post-estreno.
<code>title</code>	No se excluye: se vectoriza con TF-IDF usando unigramas y bigramas.
<code>directors</code>	No se excluye: se convierte a <code>directors_text</code> y se codifica con <code>CountVectorizer</code> .
<code>main_cast</code>	No se excluye: se convierte a <code>cast_text</code> y se codifica con <code>CountVectorizer</code> .

<code>plot,</code>	<code>genres,</code>	El scraper puede recolectar estas columnas, pero no forman parte
<code>runtime,</code>		del <code>training_dataset.json</code> usado por el modelo actual. Quedan
<code>certificate,</code>		como trabajo futuro para mejorar el feature engineering.
<code>metascore</code>		

5.1.1. Feature engineering del entrenamiento actual

El documento inicial del proyecto describe una versión de entrenamiento local basada en `training_dataset.json`. En esa versión, `train.py` aplica tres transformaciones principales antes de entrenar. Además, filtra registros sin director, convierte `reviews`, `year` y `votes` a valores numéricos, e imputa y escala las features numéricas dentro del pipeline.

La primera transformación es `log1p(votes)`. La cantidad de votos que recibe una película en IMDb tiene una distribución muy sesgada: unas pocas películas populares acumulan millones de votos, mientras que la mayoría tiene muchos menos. Esa asimetría perjudica a modelos lineales como ElasticNet. Aplicar `log1p` comprime la escala y hace que la variable sea más manejable sin perder poder predictivo.

La segunda transformación convierte las listas de directores y actores en representaciones vectoriales binarias. En el dataset original, `directors` y `main_cast` son listas de strings. Para usarlas en scikit-learn, se concatenan con el separador `|` y luego un `CountVectorizer` binario con un tokenizer por pipes genera una columna por director o actor que aparece al menos en dos películas. El resultado es un encoding multi-hot.

La tercera transformación vectoriza el título de la película con TF-IDF usando unigramas y bigramas. El título se trata como texto libre y se calculan pesos TF-IDF sobre los 1000 términos más frecuentes, considerando palabras individuales y pares de palabras consecutivas. Esto permite capturar patrones implícitos en el nombre de la película, como referencias a sagas, géneros o épocas.

Vale aclarar que esa versión del dataset no incluye algunas columnas que el scraper sí recolecta, como géneros, metascore, certificado de contenido o argumento. Esas features quedan fuera del modelo actual y representan una oportunidad clara de mejora para iteraciones futuras.

5.2. Entrenamiento del modelo

El entrenamiento actual toma como entrada el archivo local:

```
training/training_dataset.json
```

Como mejora MLOps, el entrenamiento debería leer ese dataset procesado desde S3 o registrar explícitamente qué versión local se utilizó, para mantener trazabilidad entre datos, corrida de entrenamiento y modelo publicado.

El dataset se separa en variables predictoras y target luego de construir las features derivadas:

```
features = [  
    "title",  
    "year",  
    "log_votes",  
    "reviews",  
    "directors_text",  
    "cast_text",  
]  
X = df[features]  
y = df["imdb_rating"]
```

Como primera aproximación se pueden usar modelos de regresión como Linear Regression, Random Forest Regressor, Gradient Boosting Regressor, XGBoost o LightGBM. En la implementación documentada inicialmente, el modelo entrenado es un ElasticNet dentro de un Pipeline de scikit-learn.

El script `train.py` lee la data procesada, aplica feature engineering y busca los mejores hiperparámetros `alpha` y `l1_ratio` mediante `GridSearchCV` con validación cruzada de 5 folds, optimizando MAE. Luego evalúa sobre un test set del 20 % y reporta MAE, RMSE y R^2 . Además, imprime las mejores y peores predicciones individuales para facilitar el análisis de errores.

Dentro del folder de training, en el README.MD file, se indican las instrucciones para correr

el script de entrenamiento y setear variables de ambiente que permiten hacer el upload del modelo en W&B logeando de forma transparente al usuario a dicho repositorio de modelos.

Al ejecutar el entrenamiento del ElasticNet, se imprimen en consola los siguientes datos:

```
Métricas en test:
MAE : 0.1086124870096499
RMSE: 0.138602053695697
R2 : 0.07743975577053108

Interpretación MAE:
Error promedio en escala 0-1: 0.1086
Error promedio equivalente en escala 1-10: 1.09 puntos

Peores predicciones:
      title      y_real      y_pred      abs_error
902  Allumeur-Marche      0.10      0.575188      0.475188
1775  Special Angelz      0.24      0.660575      0.420575
124   Revelation 13      0.25      0.665905      0.415905
1018  Harlan and Fiona      0.31      0.683799      0.373799
266   The 30th Annual People's Choice Awards      0.32      0.671204      0.351204
1131  Cop Corruption      1.00      0.659083      0.340917
1855  Bull Fight, No. 1      0.28      0.613088      0.333088
1716  Untitled Dystopia      0.34      0.666868      0.326868
324   Sovia              0.35      0.667549      0.317549
1936  Ayin Karanlik Yüzu      0.34      0.652503      0.312503

Mejores predicciones:
      title      y_real      y_pred      abs_error
1567  Herbstzeit         0.64      0.640462      0.000462
1050   Pepe Conde         0.61      0.607562      0.002438
1067   Bite the Big Apple      0.67      0.667332      0.002668
1870  El ragol el-lazi fakad zilloh      0.62      0.623179      0.003179
739   Roaming Lady         0.61      0.606817      0.003183
1412  Faust i Piteä        0.66      0.663340      0.003340
333   Sonetäula           0.66      0.656551      0.003449
252   Snow White: A Tale of Terror      0.61      0.613753      0.003753
1536  The Big White        0.63      0.633775      0.003775
1115  Alligator            0.61      0.606178      0.003822
```

Figura 3: training logs

En donde se indica el error promedio MAE, el error en escala 1-10 puntos(1.09 para el entrenamiento que se muestra), y un rank de las mejores y peores predicciones en test. Finalmente en W&B se publicará el modelo, pero solo será taggeado como Production si el modelo subido tiene el menor MAE entre los modelos que forman parte del registry, caso contrario solo será taggeado como latest.

5.3. Publicación en W&B

La parte más importante del cierre del entrenamiento es la publicación en Weights & Biases [6]. El script consulta el MAE del modelo actualmente en producción antes de decidir si promover el nuevo modelo. Solo asigna el alias `production` si el nuevo modelo mejora al existente; en caso contrario, lo sube únicamente con `latest`. Esto evita que un reentrenamiento sobre los mismos datos degrade el modelo productivo.

El equipo cuenta con un team en W&B, lo que permite centralizar los modelos fuera de cuentas

personales y facilita que todos los integrantes puedan subir, consultar y promover artefactos.

Configuración del registry:

```
Entity: mlprod-obli
Project: imdb-rating
Artifact: imdb-rating-model
Alias: production
```

La idea es usar alias para promover modelos:

Alias	Uso
candidate	Modelo recién entrenado.
staging	Modelo validado para pruebas.
production	Modelo usado por la API.

5.4. Prevención de data leakage y training-serving skew

En un sistema de Machine Learning en Producción no alcanza con que el modelo tenga buenas métricas offline. También es necesario asegurar que el modelo no aprenda información que no estaría disponible al momento de predecir y que el procesamiento usado en entrenamiento sea el mismo que se usa en inferencia.

5.4.1. Data leakage

Data leakage ocurre cuando el modelo usa, directa o indirectamente, información del futuro o información demasiado cercana al target. Esto genera métricas artificialmente buenas durante entrenamiento, pero mal desempeño en producción.

Riesgo	Prevención
Usar <code>imdb_rating</code> como feature	Separar explícitamente <code>X</code> e <code>y</code> , dejando <code>imdb_rating</code> solo como target.

Usar <code>votes</code> como predictor	En el modelo actual se usa transformado como <code>log_votes</code> . Si la predicción representa una película nueva o temprana, debería excluirse o reemplazarse por una señal disponible en ese momento.
Usar reviews posteriores al momento de predicción	Definir fecha de corte o escenario de predicción.
Mezclar duplicados entre train y test	Deduplicar por <code>imdb_id</code> antes del split.
Hacer transformaciones mirando todo el dataset	Ajustar imputadores, escaladores y encoders solo con train.
Elegir modelo mirando el test final	Usar train/validation para selección y reservar test para evaluación final.

Para reviews, una decisión importante es distinguir entre predicción pre-estreno o temprana, donde no deberían usarse reviews de usuarios, y predicción post-estreno, donde pueden usarse siempre que se registre la fecha de corte.

5.4.2. Training-serving skew

Training-serving skew ocurre cuando los datos que recibe el modelo en producción no pasan por el mismo procesamiento que los datos usados para entrenar. Para reducir este riesgo, la aplicación en producción no reconstruye features manualmente. En su lugar, la UI muestra una lista de películas disponibles y, cuando el usuario selecciona una, la API obtiene los datos ya procesados desde `inference_dataset.json`.

Ese JSON de inferencia debería generarse con el mismo ETL que produce `training_dataset.json`. Por lo tanto, las columnas derivadas, codificaciones, agregaciones de reviews, tratamiento de nulos y tipos de datos nacen del mismo proceso usado para entrenar.

5.5. Trazabilidad de ML

La trazabilidad permite reconstruir qué datos, código, parámetros y modelo participaron en una predicción o en una versión publicada. En este proyecto se considera versionar experimentos, modelos y datos.

Elemento	Cómo se versiona	Para qué sirve
Experimentos	Corridas en W&B con parámetros, métricas y artefactos asociados	Comparar modelos, justificar la elección final y auditar resultados.
Modelos	Artifacts de W&B con versiones y aliases	Saber qué modelo está desplegado y volver a una versión anterior.
Datos crudos	Parquet en S3 particionados por fecha y corrida	Identificar de qué extracción provienen los datos.
Dataset procesado	<code>training_dataset.json</code> generado por el ETL	Relacionar cada entrenamiento con el dataset exacto utilizado.
Datos de inferencia	<code>inference_dataset.json</code> generado por el ETL	Asegurar consistencia entre entrenamiento e inferencia.

Para que la trazabilidad sea completa, cada corrida de entrenamiento debería guardar el dataset usado, fecha y versión del ETL, parámetros del modelo, métricas, commit de Git, artifact del modelo y lista de features esperadas.

6. Servicio/API

La capa de servicio expone el modelo entrenado para ser consumido desde una aplicación web y desde endpoints HTTP. Está implementada con FastAPI, carga el modelo productivo desde W&B y utiliza datos procesados para devolver predicciones de rating de forma consistente con el entrenamiento.

6.1. API de inferencia

La aplicación de serving está implementada con FastAPI [1]. Actualmente expone endpoints para:

Endpoint	Descripción
/	Interfaz web.
/status	Health check de la aplicación.
/movies	Lectura de películas desde S3.
/model	Consulta del artifact productivo en W&B.
POST /predict	Recibe una película seleccionada o su <code>imdb_id</code> , busca features en el JSON de inferencia y devuelve el rating estimado.

El endpoint de predicción carga el modelo productivo desde W&B, recupera del JSON de inferencia las features ya procesadas, excluye columnas que no deben usarse como entrada del modelo y devuelve una respuesta consistente.

6.2. Predicciones en landing page

Para hostear la landing page se optó por usar AWS ECS sobre AWS Academy [5]. Detrás de un Application Load Balancer se ejecuta una task construida desde una imagen publicada en AWS ECR, la cual se pusha localmente usando AWS CLI.

Antes de servir la imagen a ECR, se construye localmente a partir del `Dockerfile`. La imagen contiene el contenido estático de la página web y el web service que oficia de backend, utilizando

JavaScript en el frontend y Python FastAPI como web engine.

Una vez levantada la task en ECS, la URL del Load Balancer expone el servicio en Internet. El servicio se conecta a W&B para descargar el modelo y hacer inferencia. Además, obtiene películas que ofician de input para la inferencia desde TMDb, sitio similar a IMDb que expone una API gratuita. Las películas se cargan a demanda desde una barra de búsqueda y, al seleccionar una película, se compara el score predicho contra el score real.

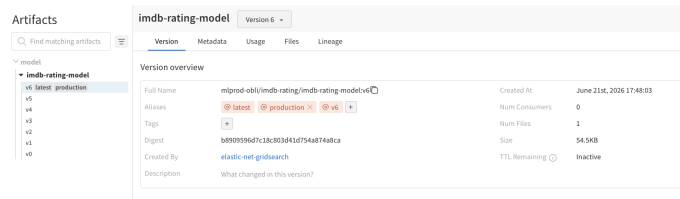


Figura 4: W&B Model Registry

Las películas se cargan a demanda en la barra de búsqueda



Figura 5: Enter Caption

Luego al seleccionar una película, se hace la predicción y se compara el score predicho contra el score real de la película en TDMB

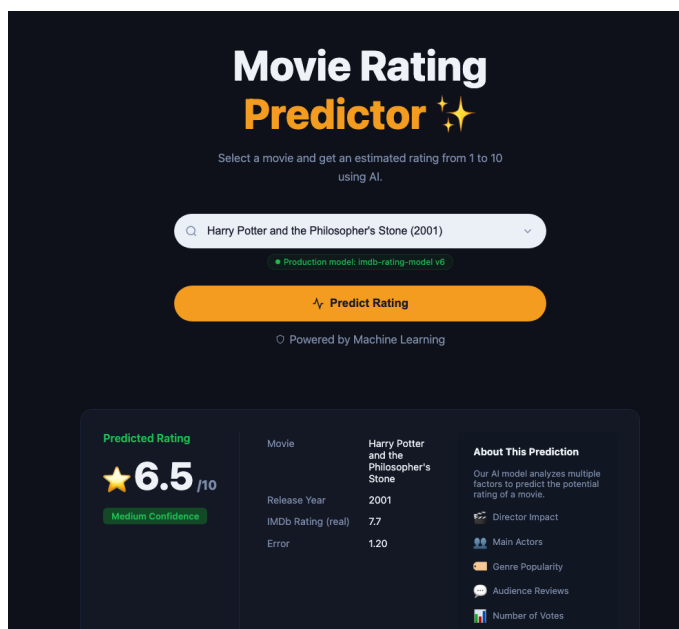


Figura 6: Ejemplo de predicción - Harry Potter(2001)

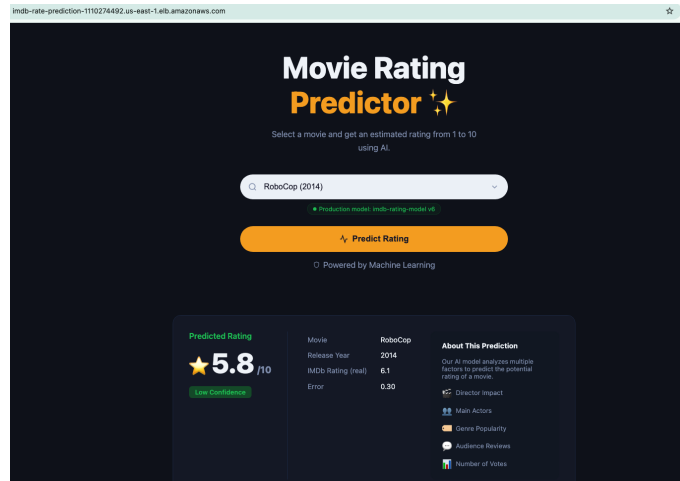


Figura 7: Ejemplo de predicción - Robocop(2014)

6.3. Predicciones batch

Además de predicciones individuales por medio de la landing page, se deja planteado exponer predicciones en modo batch. Esta funcionalidad permitiría recibir un conjunto de películas, ejecutar inferencia sobre todas ellas y devolver un archivo o respuesta agregada con los ratings estimados. Para esto, el pod de ECS hostea un endpoint(/predict-batch) en donde dada una lista de identificadores de películas extraídos de TMDb, por ejemplo, "tmdb_ids" : [27205, 155, 550], el servicio retorna las predicciones para cada película incluyendo el rate real de la película en IMDB.

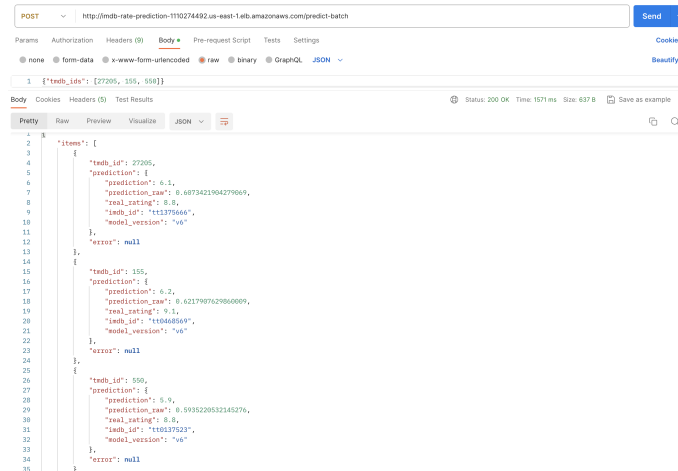


Figura 8: Predicción en Batch

6.4. Containerización

La aplicación se empaqueta con Docker usando una imagen base de Python. El contenedor instala las dependencias, copia la aplicación FastAPI y expone el puerto 8000. Esto permite ejecutar el mismo artefacto en local y en producción, reduciendo diferencias entre ambientes.

```
docker compose up --build
```

7. Infraestructura y monitoreo

La infraestructura productiva para el hosting de la Landing Page se define sobre AWS y busca que el despliegue sea repetible, observable y separado de la configuración sensible. En este bloque se agrupan la definición de infraestructura, la gestión de secretos, los mecanismos de monitoreo y la automatización operativa del proyecto.

7.1. Infraestructura como código

La infraestructura cloud se define con Terraform. Esto permite versionar los recursos, recrearlos y documentar la arquitectura de despliegue.

Recursos definidos:

- AWS ECS Cluster.
- Task Definition para Fargate.
- ECS Service.
- Application Load Balancer.
- Target Group con health check en `/status`.
- Security Groups.
- CloudWatch Log Group.
- Política de acceso a S3 para la task.

Para hostear la landing page se optó por usar AWS ECS sobre AWS Academy, en donde detrás de un Load Balancer vive un pod que es construido mediante una imagen publicada en AWS ECR, la cual se pusha localmente usando el AWS CLI.

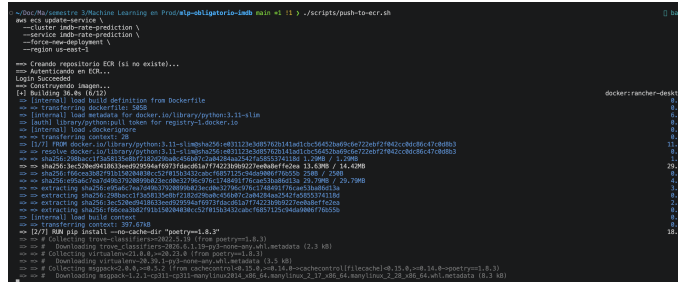


Figura 9: Build local de la imagen Docker

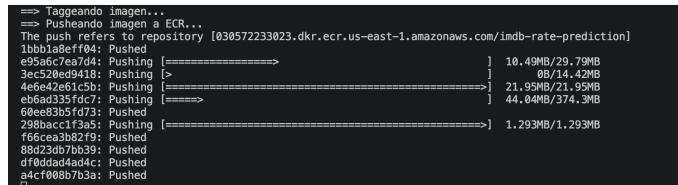


Figura 10: Push de la imagen Docker hacia AWS ECR

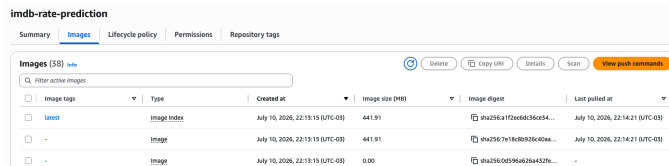


Figura 11: imagen de ECR pusheada como latest

Para construir componentes en la nube(IaC), la herramienta elegida fue Terraform, la cual usando las claves de la cuenta de AWS Academy es capaz de crear, actualizar y destruir recursos en dicha cuenta.

A partir del script **push-to-ecr.sh**, se crea el repositorio de imágenes dentro de ECR, luego se buildea la imagen y se pushea a dicho repositorio. Acto seguido, se procede a correr el comando "terraform apply" para construir el resto de la infra(incluido el repositorio de imágenes si es que no existía anteriormente).

```

aws_cloudwatch_log_group.app: Creating...
aws_s3_bucket_policy.app: read: Creating...
aws_ecs_cluster.app: Creating...
aws_security_group.alb: Creating...
aws_lb_target_group.app: Creating...
aws_s3_bucket_policy.app: read: Creation complete after 1s [id:aws-test-bucket-2026]
aws_cloudwatch_log_group.app: Creation complete after 1s [id:aws-lbmb-rate-prediction]
aws_ecs_task_definition.app: Creating...
aws_ecs_task_definition.app: Creation complete after 1s [id:aws-lbmb-rate-prediction]
aws_lb_target_group.app: Creation complete after 3s [id:aws-mas-elasticloadbalancing-us-east-1-638572233823-targetgroup/lbmb-rate-prediction/3c221b3ec4561cc2]
aws_security_group.alb: Creation complete after 4s [id:aws-test-763240906de]
aws_lb.app: Creating...
aws_security_group.app: Creating...
aws_security_group.app: Creation complete after 5s [id:aws-8cd6fc6639ac2261]
aws_ecs_cluster.app: Still creating... [10s elapsed]
aws_ecs_cluster.app: Creation complete after 12s [id:aws-mas-ecs-us-east-1-638572233823-cluster/lbmb-rate-prediction]
aws_lb.app: Still creating... [10s elapsed]

```

Figura 12: Terraform apply - Se crean los recursos

Finalmente, cuando la task se encuentra running" sabemos que la Landing Page ya se encuentra operativa.



Figura 13: Task de ECS ya levantada

7.2. Gestión de secretos

La API necesita credenciales para consultar W&B y autenticarse contra TMDB. Para evitar hardcodear secretos, se usa AWS SSM Parameter Store.

```

wandb-org
wandb-api-key
omdb-api-key
tmdb-token

```

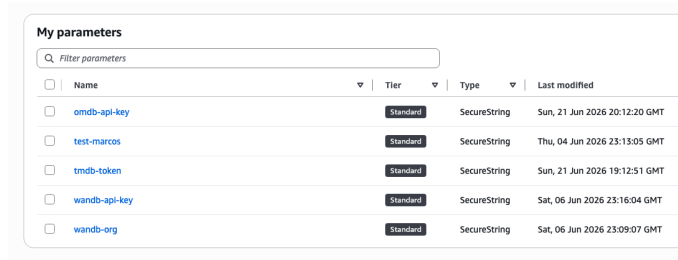


Figura 14: AWS Parameter Store

Si no se encuentran en SSM, la aplicación permite usar variables de entorno como fallback:

```
WANDB_USER
WANDB_API_KEY
TMDB_TOKEN
OMDB_API_KEY
```

Esto aplica una buena práctica de seguridad: separar secretos del código fuente.

7.3. Observabilidad y monitoreo

La solución incorpora observabilidad en dos niveles: monitoreo operativo del servicio desplegado y monitoreo funcional de las predicciones generadas por el modelo. Esto permite revisar si la API está viva, qué versión de modelo está sirviendo y cómo se comportan las predicciones frente al rating real disponible.

Componente	Uso en el proyecto
<code>/status</code>	Health check de FastAPI. Devuelve si la aplicación está activa y si el modelo productivo fue cargado correctamente. También se usa como health check del Target Group en el Application Load Balancer.
<code>/model</code>	Endpoint para consultar el artifact productivo en W&B, incluyendo versión, alias, fecha de creación y URL. Sirve para verificar qué modelo está siendo referenciado por el servicio.
Logs de ECS	Los contenedores envían logs a CloudWatch. Allí quedan registrados eventos de startup, carga de secretos desde SSM, descarga del modelo desde W&B, errores de conexión con APIs externas y errores de inferencia.
Eventos de predicción	Cada llamada a <code>/predict</code> o <code>/predict-batch</code> genera un evento de predicción mediante <code>monitoring/logger.py</code> . El evento se guarda como JSON en S3 bajo el prefijo <code>predictions/</code> .

Dashboard de monitoreo `monitoring/analyze.py` implementa un dashboard en Streamlit que lee esos eventos desde S3 y muestra métricas agregadas, gráficos y predicciones recientes.

Los eventos de predicción guardan información útil para análisis posterior: `timestamp`, `tmdb_id`, `imdb_id`, título, predicción, rating real cuando está disponible, error absoluto, versión del modelo, estrategia del modelo, cantidad de reviews usadas, score VADER y `log_votes`. Se guarda un archivo por predicción, lo que evita problemas de concurrencia al escribir desde varias llamadas simultáneas.

El dashboard de Streamlit permite observar el comportamiento del modelo en producción de forma exploratoria. Muestra el total de predicciones, MAE contra el rating real de IMDb/OMDb cuando está disponible, porcentaje de predicciones con error menor a un punto, cantidad de versiones de modelo vistas, dispersión entre predicción y valor real, distribución de errores, MAE móvil por ventana temporal, comparación de MAE por versión de modelo y las predicciones más recientes.

La infraestructura del dashboard también está definida con Terraform dentro de `infra/observability`. Se crea un repositorio ECR para la imagen de monitoreo, un bucket S3 de observabilidad, un cluster/servicio ECS Fargate para Streamlit, un security group que expone el puerto 8501 y un log group de CloudWatch con retención de 7 días.

Como decisión práctica para el obligatorio, el bucket de observabilidad se configuró con acceso público de lectura/escritura para simplificar la escritura de eventos desde la API y la lectura desde el dashboard, evitando depender de credenciales temporales en esa parte del flujo. En un entorno productivo real, esta decisión debería reemplazarse por permisos IAM más restrictivos, autenticación del dashboard y una política de retención de datos.

Limitaciones y mejoras pendientes:

- El dashboard es de lectura manual; todavía no dispara alertas automáticas ante drift, errores altos o caída del servicio.
- No se registra latencia por endpoint ni métricas HTTP agregadas como tasa de errores o

percentiles de respuesta.

- El drift se analiza de forma indirecta a partir de features registradas y errores observados, pero no hay tests automáticos de drift contra el dataset de entrenamiento.
- La API carga el modelo al iniciar; si cambia el alias **production** en W&B, hace falta reiniciar o redeplegar el servicio para asegurar que use la nueva versión.

8. Limitaciones del proyecto y arquitectura ideal

8.1. Limitantes

Más allá de las limitantes puntuales ya mencionadas en la recolección de datos, el proyecto tuvo una restricción transversal que condicionó buena parte de las decisiones de arquitectura, se trabajó con cuentas individuales de AWS Academy/Learner Lab en lugar de una cuenta o una organización compartida entre los integrantes del equipo.

Esto trajo varias consecuencias prácticas:

- No fue posible compartir directamente buckets de S3, repositorios de ECR ni los servicios de ECS entre integrantes, ya que cada cuenta de Academy es aislada y temporal. Quien avanzaba con una parte de la infraestructura terminaba alojando esos recursos en su propia cuenta.
- El estado de Terraform no pudo centralizarse en un backend remoto compartido, por lo que aplicar infraestructura quedó atado a la persona que la había creado originalmente.
- Los despliegues a ECR/ECS se hicieron de forma manual (`scripts/push-to-ecr.sh`, `terraform apply local`) en lugar de a través de un pipeline de CI/CD, porque no había un rol de despliegue ni credenciales de larga duración que todo el equipo pudiera usar de forma segura.
- Las credenciales temporales de Academy expiran en pocas horas, lo que además de limitar el volumen de scraping impidió dejar corriendo procesos programados (schedulers, triggers automáticos) que dependieran de esas credenciales.
- El gate de promoción de modelos que compara MAE contra producción (ver Publicación en W&B) corre hoy de forma manual/local al ejecutar `train.py`, y aunque promueva el alias `production` en W&B, la API en ECS solo carga el modelo una vez al iniciar (`deployment/main.py`), por lo que hace falta reiniciar el servicio a mano para que sirva el modelo nuevo.

8.2. Arquitectura ideal

La Figura 15 propone cómo debería quedar la arquitectura si el proyecto se llevara a un entorno real, siguiendo los principios de Continuous Delivery for Machine Learning (CD4ML) [2]. Los componentes marcados en azul con un check ya están implementados en la solución actual, mientras que los marcados en naranja con línea punteada son las piezas que faltarían para alcanzar una solución end to end.

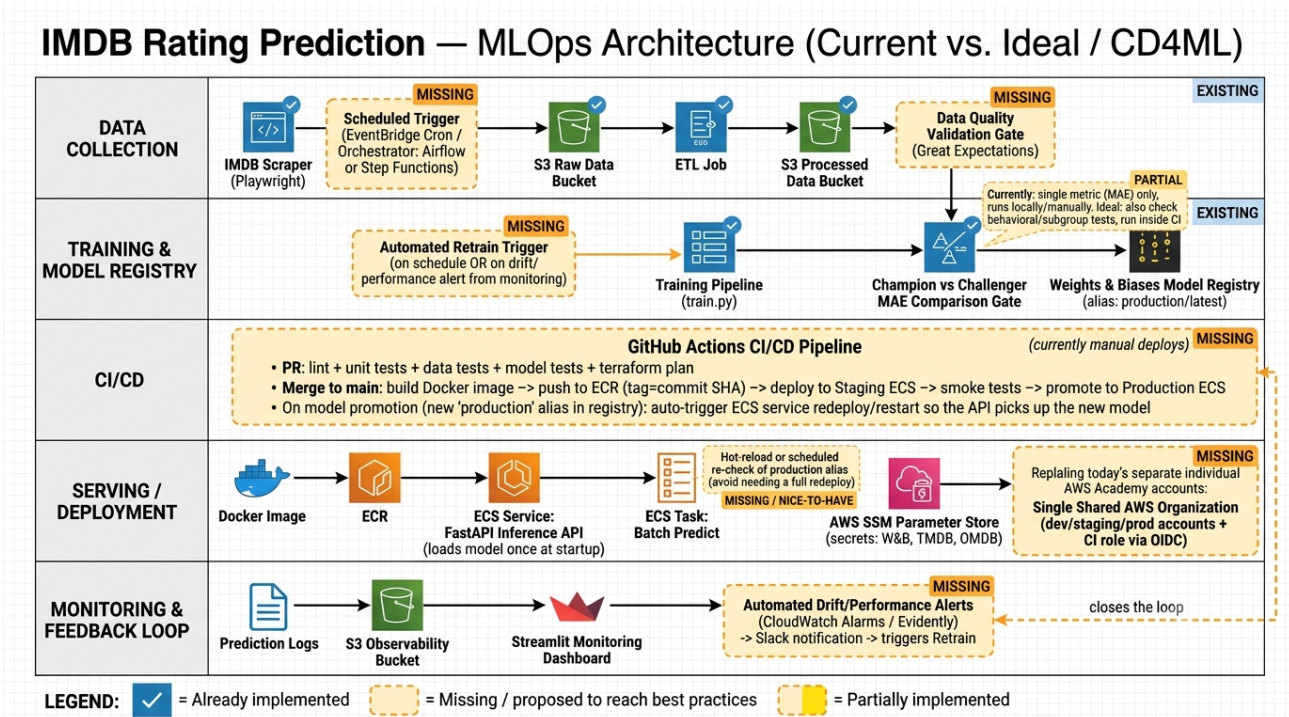


Figura 15: Arquitectura ideal propuesta (CD4ML): componentes ya implementados versus componentes faltantes para una solución MLOps completa.

A partir de esa comparación, las mejoras principales para un proyecto ideal serían:

- **Cuenta/organización de AWS compartida.** Reemplazar las cuentas individuales de Academy por una AWS Organization con cuentas separadas por ambiente (dev/staging/prod) y un rol de CI/CD asumible vía OIDC, de forma que el equipo completo pueda compartir buckets, ECR y el estado de Terraform sin depender de una única persona.

- **Recolección de datos programada.** Disparar el scraper y el ETL automáticamente (Event-Bridge, Step Functions o un orquestador como Airflow) en lugar de ejecutarlos manualmente, y agregar un gate de validación de calidad de datos (por ejemplo con Great Expectations) antes de habilitar un dataset para entrenar.
- **Reentrenamiento automático.** Disparar el pipeline de entrenamiento por schedule o por evento (nuevos datos suficientes, degradación detectada en el monitoreo), en lugar de requerir que alguien lo corra a mano.
- **Fortalecer el gate de promoción de modelos.** El gate actual (comparar MAE contra el modelo en producción) es un buen punto de partida; se podría extender con tests de comportamiento y por subgrupos, y moverlo a un pipeline de CI en lugar de correr localmente.
- **Redeploy automático al promover un modelo.** Que la promoción del alias `production` en W&B dispare automáticamente un redeploy o reinicio del servicio de ECS, o alternatively que la API chequee periódicamente si hay un nuevo alias productivo, evitando servir un modelo desactualizado.
- **CI/CD real con GitHub Actions.** Lint, tests unitarios, tests de datos y de modelo, y `terraform plan` en cada PR; build de imagen, push a ECR con tag por commit, deploy a staging y promoción a producción automática al merge a main.
- **Cerrar el loop de monitoreo.** Convertir el dashboard de Streamlit, que hoy es de lectura manual, en una fuente de alertas automáticas (CloudWatch Alarms o herramientas como Evidently) que creen notificaciones y puedan disparar un reentrenamiento cuando se detecte drift o degradación de performance.

9. Conclusiones

El proyecto permitió construir una solución de Machine Learning en Producción de punta a punta para estimar ratings de películas. El resultado no se limita al entrenamiento de un modelo, sino que integra las piezas necesarias para operar un sistema de ML: recolección de datos, almacenamiento en S3, procesamiento ETL, feature engineering, tracking de experimentos, registry de modelos, API de inferencia, despliegue cloud, gestión de secretos, infraestructura como código y una primera capa de monitoreo.

Uno de los aprendizajes principales fue que, en un sistema productivo, el modelo es solo una parte del problema. Gran parte del esfuerzo estuvo en hacer que los datos fueran trazables, que el procesamiento pudiera repetirse, que el modelo publicado pudiera identificarse mediante W&B y que la API sirviera predicciones de forma consistente con el entrenamiento. En ese sentido, la separación entre datos crudos y procesados, el uso de Parquet en S3, la generación de datasets de entrenamiento e inferencia, y el empaquetado con Docker aportan una base concreta para reducir deuda técnica y training-serving skew.

También se logró llevar el modelo a un entorno cloud usando AWS ECS Fargate, ECR, Application Load Balancer, Terraform y Parameter Store. Esto permitió ejercitar decisiones propias de MLOps en producción, cómo publicar una imagen, inyectar secretos sin hardcodearlos, exponer health checks, consultar el modelo productivo desde W&B y registrar eventos de predicción para análisis posterior. La incorporación del dashboard de monitoreo en Streamlit complementa la solución al permitir observar errores, versiones servidas y comportamiento reciente del modelo.

Como cierre, la siguiente tabla resume los principales conceptos de clase aplicados en el proyecto:

Concepto	Aplicación en el proyecto
Pipeline de datos	Scraper + S3 + ETL.
Data lake	S3 como almacenamiento central.
Versionado de datos	Archivos particionados por fecha y corrida.
Feature engineering	Generación de features numéricas, categóricas y textuales.

Prevención de data leakage	Exclusión de variables no disponibles o demasiado cercanas al target.
Training-serving skew	Pipeline y schema compartidos entre entrenamiento e inferencia.
Reproducibilidad	Docker, Terraform y Makefile.
Tracking de experimentos	W&B para métricas y parámetros.
Model registry	Artifacts y aliases en W&B.
Trazabilidad de ML	Versionado de experimentos, modelos, datos y features usadas.
Serving	FastAPI como API de inferencia.
Predicción batch	Endpoint para inferir sobre múltiples películas en una misma solicitud.
CI/CD potencial	Build de imagen, push a ECR y deploy con Terraform ejecutados de forma manual/local.
Gestión de secretos	AWS SSM Parameter Store.
Observabilidad	Health check, CloudWatch logs, eventos de predicción en S3 y dashboard en Streamlit.
Despliegue cloud	ECS Fargate + ALB.

Las principales limitaciones estuvieron asociadas al contexto académico y operativo, como el uso de cuentas individuales de AWS Academy, credenciales temporales, ausencia de un backend remoto compartido para Terraform, ejecución manual del scraper/ETL/entrenamiento y necesidad de reiniciar el servicio para tomar un nuevo modelo promovido a **production**. Además, algunas features recolectadas por el scraper, como géneros, duración, certificado, metascoring y sinopsis, todavía no se incorporan al dataset final del modelo, por lo que existe margen claro para mejorar la capacidad predictiva.

Como trabajo futuro, la evolución natural sería cerrar el ciclo CD4ML, automatizando la recolección y validación de datos, entrenar desde datasets versionados en S3, ejecutar tests de datos y de modelo en CI, promover modelos mediante gates más completos, redeployar automáticamente

cuando cambia el modelo productivo y transformar el monitoreo actual en alertas accionables ante drift o degradación de performance.

En síntesis, la solución implementada cumple el objetivo central del obligatorio, demostrar una arquitectura MLOps funcional que conecta datos, modelo e infraestructura. Aunque todavía no alcanza el nivel de automatización de una plataforma productiva ideal, deja implementados los componentes principales y documenta claramente qué decisiones deberían fortalecerse para llevarla a un entorno real.

Referencias

- [1] FastAPI. Fastapi documentation. <https://fastapi.tiangolo.com/>, 2026.
- [2] Martin Fowler. Continuous delivery for machine learning. <https://martinfowler.com/articles/cd4ml.html>, 2016.
- [3] Chip Huyen. *Designing Machine Learning Systems*. O'Reilly Media, 2022.
- [4] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 2015.
- [5] Amazon Web Services. Amazon elastic container service documentation. <https://docs.aws.amazon.com/ecs/>, 2026.
- [6] Weights and Biases. Weights and biases documentation. <https://docs.wandb.ai/>, 2026.